

GraphOne / FrontierAtlas: Technical Architecture Specification

This specification outlines the fault-tolerant, high-throughput intelligence pipeline engineered to ingest, normalize, and synchronize market intelligence across research papers, startups, products, news, and jobs.

Section 1: Scale Strategy (Horizontal Scaling to 500,000+ Entities)

To scale acquisition and normalization to 500,000+ entities daily, the ingestion architecture abstracts concurrency from localized scripts into an asynchronous producer-consumer pipeline designed for horizontal scale.

- **Decoupled Producer-Consumer Concurrency:** The pipeline operates on an asynchronous multi-worker model (`asyncio`), using 10 concurrent worker tasks coordinated by a producer orchestrator (`src/main_phase1.py`, `src/main_phase2.py`). Workers pull extraction tasks independently, preventing slow endpoints from bottlenecking overall throughput. In a production cluster, this queue structure migrates to distributed message brokers (such as Redis Streams or RabbitMQ) across Kubernetes worker pods.
 - **Domain Sharding & Anti-Bot Fingerprinting:** Ingestion targets are sharded by host domain to enforce domain-specific rate limits, connection parameters, and evasion profiles:
 - **Directory Ingestion (There's An AI For That):** Handled via `curl_cffi` using `AsyncSession` with Chrome 120 browser impersonation (`impersonate="chrome120"`), full `Sec-Ch-Ua` headers, session warmup requests to acquire Cloudflare cookies, and randomized page delays (2.0s–5.0s) to bypass Turnstile barriers.
 - **API Ingestion (ArXiv):** Dedicated client sessions manage XML API endpoints with batch pagination (100 records per query) and controlled pacing to prevent rate limits.
 - **Resource Boundaries & Streamed Persistence:** To prevent Socket and File Descriptor exhaustion (`ulimit -n`), network sessions maintain persistent connection pooling. Memory footprint is strictly bounded by streaming parsed records directly to append-only JSONL files (`data/processed/products.jsonl`, `research_papers.jsonl`, `jobs.jsonl`, `startups.jsonl`, `news.jsonl`) rather than retaining large batch arrays in heap memory.
-

Section 2: Resilient Extraction Engine (Handling 301s, 429s, and Anti-Bot Gates)

To eliminate extraction hallucinations and runtime failures, the ingestion layer relies on deterministic parsing, protocol adherence, and defensive retry mechanics rather than volatile, ungrounded external LLM calls.

- **DOM Pruning & High-Performance Lexical Parsing:** Raw HTML from web sources is parsed using the C-based Lexbor engine via `selectolax (HTMLParser)`. Unnecessary DOM nodes (`<script>`, `<style>`, SVGs, navigation bloat) are stripped out, and CSS selectors directly target semantic content containers (e.g., `li.li`, `.name`, `div.paper-card`).
 - **Protocol & Redirect Enforcement (HTTP 301):** Legacy HTTP endpoints (such as `http://export.arxiv.org`) return 301 Moved Permanently. The crawler explicitly enforces HTTPS transport (`https://export.arxiv.org/api/query`) across all requests, eliminating redirect overhead and preventing redirection failures in asynchronous clients.
 - **Rate-Limit Backoff (HTTP 429):** When high-volume query endpoints return 429 Too Many Requests (common under burst queries on ArXiv), the crawler captures the response status and applies a defensive sleep backoff (30s) before retrying the exact un-incremented `start_index`, preventing data dropouts during batch pagination.
 - **Deterministic Normalization (Zero-Hallucination):**
 - **Pricing Models:** The `map_pricing()` engine extracts pricing from raw DOM strings using rule-based lexical matching (`100% free -> FREE`, `freemium / free trial -> FREEMIUM`, `paid / price / $ -> PAID`), defaulting unlisted items to `UNSPECIFIED` rather than guessing.
 - **GitHub Hydration:** ArXiv paper abstracts (`atom:summary`) are parsed via regular expressions to identify published repository links. Extracted repositories are hydrated with star metrics (`GitHubEnricher.get_stars()`) via the GitHub REST API.
-

Section 3: Signal Freshness & Content-Hash Deduplication (24-Hour Guarantee)

Processing fast-moving signals—such as jobs and industry news—requires strict temporal validation and cryptographic deduplication to prevent stale data and duplicate entries.

- **Two-Stage Temporal Validation:**
 1. **Timestamp Normalization:** Published dates are parsed using `datetime.fromisoformat` into standardized UTC timestamps. Any signal older than `T - 24 hours` is filtered at the ingestion gate.
 2. **Chronological Sorting:** ArXiv API endpoints enforce chronological descending order (`sortBy=submittedDate&sortOrder=descending`) to retrieve active submissions instead of legacy archival papers.

- **Composite Content Hashing:** Job boards often append dynamic tracking queries (`?ref=...`, click IDs) to URLs, causing identical job postings to generate different URL hashes. The pipeline bypasses raw URL hashing entirely, computing a SHA-256 composite hash over normalized core entity fields: `Hash = SHA-256(lower(Job Title) | lower(Company Name))`
 - **Stateful SQLite Deduplication:** Hashes are evaluated against a local SQLite database (`staging.db`, `sqlite_store.py`). Insertions use transactional `ON CONFLICT DO NOTHING` semantics, ensuring deduplicated role payloads across job boards (e.g., ArbeitNow, Himalayas).
-

Section 4: Dual Storage Strategy & Google Sheets Synchronization

The pipeline employs a staged storage architecture designed for local validation, cloud synchronization, and future relational/graph querying.

- **Staging Layer (JSONL + SQLite):** Raw crawler extractions are validated against schema models (`src/models/schemas.py`) and stored as line-delimited JSON (`.jsonl`). An SQLite transactional buffer tracks signal hashes and deduplication state across pipeline phases.
- **Entity Resolution & Cross-Vertical Mapping:** Entity linking is executed across startups, products, jobs, and news articles. The `Entity Mapping Log` documents resolutions, tracking match types:
 - **EXACT:** Direct case-insensitive string alignment (Confidence: 1.0).
 - **FUZZY:** Normalized token and string similarity scoring (Confidence: 0.82–0.93), linking raw company names to canonical startup/product entities.
- **Atomic Google Sheets Deployment (`sheets_export.py`):** Verified data from `data/processed/` is synchronized directly to Google Sheets using Google Cloud service account authentication (`credentials.json`). To eliminate historical ghost data and stale rows, the exporter executes an explicit `worksheet.clear()` call prior to writing sanitized data arrays across all active tabs.
- **Production Storage Target (PostgreSQL + Neo4j):** In a cloud production environment:
 - **PostgreSQL:** Acts as the primary operational system of record, enforcing referential integrity (Foreign Keys) between startups, products, and jobs, with `JSONB` storage for semi-structured metadata.
 - **Neo4j / Graph Store:** Traverses multi-hop entity relationships (e.g., *Research Paper* -> *Author* -> *Startup Founder* -> *Product*), enabling network analysis across the market intelligence dataset.